OpenMP Parallelization in ConTree

Optimal Decision Tree Solver - CS406 Project

Borhan Javadian Emre Ute Harun Yilmaz
Deniz Kaan Yilmaz Tolga Toker Yusuf Demirci

Group 2

January 13, 2026

Presentation Outline

1. Problem Overview
2. Parallelization Architecture
3. Level 1: main.cpp - Thread Pool
4. Level 2: general_solver.cpp - Subtree & Feature Parallelism
5. Level 3: specialized_solver.cpp - Feature Parallelism
6. Avoiding Nested Parallelism
7. Performance Results
8. Summary
9. Room for Improvement

What is ConTree?

Optimal Decision Tree Solver

ConTree finds **provably optimal** decision trees using branch-and-bound search.

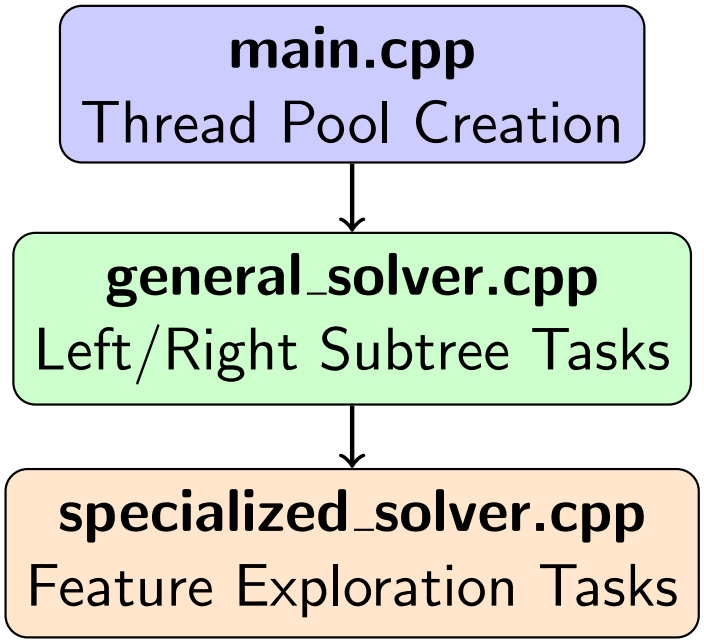
Computational Challenge:

- Exhaustive search over all features and split points
- Recursive tree construction (exponential complexity)
- Depth-2 trees solved by specialized algorithm

Goal

Parallelize the search to reduce computation time while maintaining correctness.

Three-Level Parallel Architecture



`#pragma omp parallel`

`#pragma omp task`

`#pragma omp task`

Key Design: Tasks, Not Nested Regions

All parallelism uses **OpenMP tasks** within a single parallel region.
This avoids the overhead of creating nested thread teams.

main.cpp: Creating the Thread Pool

```
// Start parallel region at top level for task-based parallelism
#pragma omp parallel
{
    #pragma omp single
    {
        GeneralSolver::create_optimal_decision_tree(
            dataview, config, optimal_decision_tree, INT_MAX);
    }
}
```

Step-by-step:

1. `#pragma omp parallel` creates a **team of threads** (default: all CPU cores)
2. `#pragma omp single` ensures only **one thread** starts the computation
3. Other threads **wait in the pool**, ready to execute tasks
4. When tasks are created later, idle threads pick them up

Why parallel + single?

Without parallel region:

- Tasks have no threads to run on
- Everything executes sequentially
- `#pragma omp task` is ignored

With parallel + single:

- Thread pool is ready
- One thread does initial work
- Tasks distributed to idle threads

Critical Insight

The parallel region is created **once at the top level**.
All nested functions use **tasks only** - no new parallel regions!

General Solver: Feature Parallelization

The general solver explores all features as potential root splits:

```
// Parallel feature exploration
for (int feature_nr = 0; feature_nr < dataview.get_feature_number(); feature_nr++) {
    int feature_index = dataview.gini_values[feature_nr].second;

    #pragma omp task firstprivate(feature_index) \
                     shared(current_optimal_decision_tree, dataview, ...)
    {
        int current_upper_bound;
        #pragma omp atomic read
        current_upper_bound = current_optimal_decision_tree->misclassification_score;

        if (current_upper_bound > 0 && solution_configuration.stopwatch.IsWithinTimeLimit
              ()) {
            create_optimal_decision_tree(dataview, ..., feature_index, ...);
        }
    }
}
#pragma omp taskwait
```

Key insight: Each feature can be explored **independently** as a task!

General Solver: Subtree Independence

For each candidate split point, we need to evaluate:

- **Left subtree**: Optimal tree for data points $\leq$ threshold
- **Right subtree**: Optimal tree for data points $>$ threshold

Key observation: Left and right are **completely independent**!

```
Dataview left_dataview = ...;   // Separate copy
Dataview right_dataview = ...;  // Separate copy

std::shared_ptr<Tree> left_optimal_dt = ...;  // Own output
std::shared_ptr<Tree> right_optimal_dt = ...; // Own output

// These two calls don't share any state!
GeneralSolver::create_optimal_decision_tree(left_dataview, ..., left_optimal_dt);
GeneralSolver::create_optimal_decision_tree(right_dataview, ..., right_optimal_dt);
```

General Solver: Parallel Subtree Tasks

```
// Task granularity cutoff: avoid overhead for small problems
const int min_dataset_size_for_parallel = 1000;
const int min_depth_for_parallel = 3;
const bool should_parallelize =
    (solution_configuration.max_depth >= min_depth_for_parallel) &&
    (dataview.get_dataset_size() >= min_dataset_size_for_parallel);

if (should_parallelize) {
    // Create task for LEFT subtree
    #pragma omp task shared(left_dataview, left_optimal_dt)
    {
        GeneralSolver::create_optimal_decision_tree(
            left_dataview, left_solution_configuration, left_optimal_dt, ...);
    }

    // Create task for RIGHT subtree
    #pragma omp task shared(right_dataview, right_optimal_dt)
    {
        GeneralSolver::create_optimal_decision_tree(
            right_dataview, right_solution_configuration, right_optimal_dt, ...);
    }

    #pragma omp taskwait  // Wait for BOTH to complete before using results
}
```

Granularity Cutoff: Why It Matters

Problem: Task Overhead

- Creating a task has cost
- Scheduling adds latency
- Small tasks: overhead $>$ benefit

Solution: Cutoff Thresholds

- Depth ≥ 3 (enough work)
- Dataset ≥ 1000 points
- Below threshold: run sequentially

Example

Situation	**Depth**	**Parallelize?**
Root problem	5	Yes (if size ≥ 1000)
Near leaves	1	No (too shallow)
Small subset	any	No (dataset too small)

Specialized Solver: Depth-2 Optimization

For depth-2 trees, we use an optimized algorithm:

- No recursion needed
- Direct computation of depth-1 subtrees
- Explores all **features** as potential root splits

Parallelization opportunity:

- Each feature can be explored **independently**
- But they all update the **same best tree** (shared state!)

Challenge

Unlike general solver, we have **shared mutable state** - the best tree found so far. This requires **synchronization**.

Specialized Solver: Feature Exploration Tasks

```
void SpecializedSolver::create_optimal_decision_tree(
    const Dataview& dataview, const Configuration& solution_configuration,
    std::shared_ptr<Tree>& current_optimal_decision_tree, int upper_bound)
{
    const int feature_count = dataview.get_feature_number();

    // Create a task for EACH feature
    for (int feature_index = 0; feature_index < feature_count; feature_index++) {
        #pragma omp task firstprivate(feature_index) \
                         shared(current_optimal_decision_tree, dataview, ...)
        {
            // Each task explores one feature as root split
            int current_score;
            #pragma omp critical(specialized_tree_access)
            {
                current_score = current_optimal_decision_tree->misclassification_score;
            }

            if (current_score > solution_configuration.max_gap) {
                create_optimal_decision_tree(..., feature_index,
                    current_optimal_decision_tree, ...);
            }
        }
    }
    #pragma omp taskwait  // Wait for all features to be explored
}
```

Critical Sections: Protecting Shared State

Problem: Multiple tasks read/write `current_optimal_decision_tree`

Solution: Use `critical` sections with a **named lock**

```
// Reading current best score (before doing work)
#pragma omp critical(specialized_tree_access)
{
    current_score = current_optimal_decision_tree->misclassification_score;
}

// Updating the tree (after finding potentially better solution)
#pragma omp critical(specialized_tree_access)
{
    // Double-check pattern: re-verify inside lock
    if (current_best_score < current_optimal_decision_tree->misclassification_score) {
        current_optimal_decision_tree->misclassification_score = current_best_score;
        current_optimal_decision_tree->update_split(feature_index, threshold, ...);
    }
    current_misclassification = current_optimal_decision_tree->misclassification_score;
}
```

Why Named Critical Sections?

Syntax: `#pragma omp critical(name)`

All critical sections with the **same name** use the **same lock**.

We use `specialized_tree_access` in three places:

1. **Line 50-53**: Read score before starting feature exploration
2. **Line 254-257**: Read score for pruning decisions
3. **Line 287-296**: Write updated tree (with double-check)

Double-Check Pattern

We re-check the condition **inside** the lock because another thread may have updated the tree between our computation and our attempt to write.

Atomic Operations for Statistics

For simple counters, use `atomic` instead of `critical`:

```
#pragma omp atomic
statistics::total_number_of_specialized_solver_calls += 1;
```

atomic

- Single memory operation
- Hardware-level guarantee
- Very low overhead
- Limited to simple operations

critical

- Arbitrary code block
- Software lock
- Higher overhead
- More flexible

The Nested Parallelism Trap

What We Tried (and Failed)

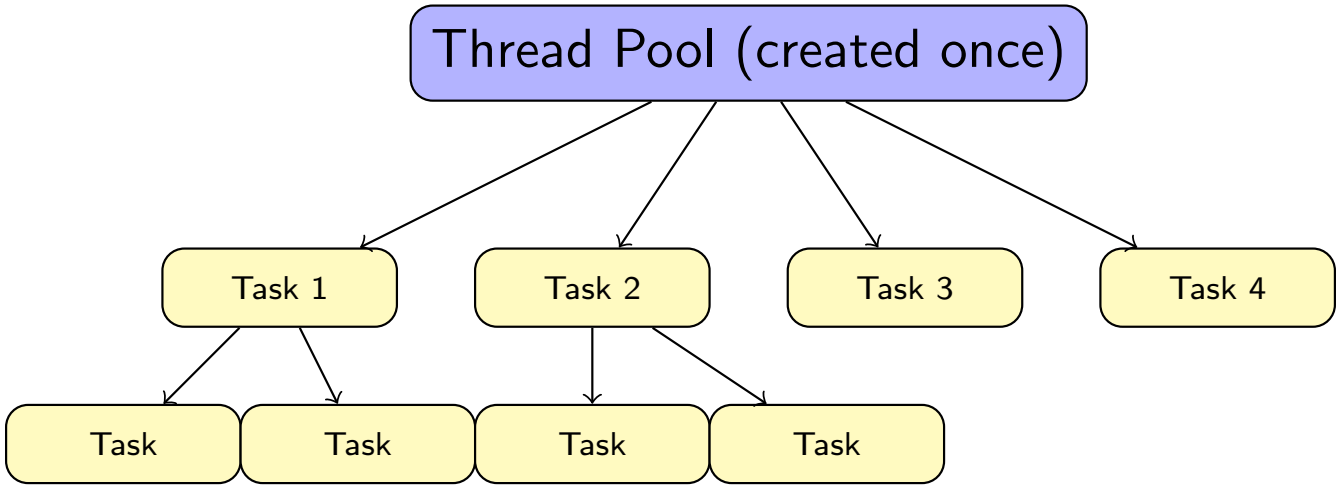
Initially, we added `#pragma omp parallel` inside specialized_solver:

- Created a **new team of threads** for each call
- When called from general_solver tasks: **nested teams**!
- Result: 4x **slower** than sequential (25s vs 6.7s)

Why It's Slow

- Thread creation is expensive
- Nested teams compete for same CPU cores
- Synchronization overhead multiplies
- Cache thrashing between thread teams

The Solution: Tasks Only



All tasks share the same thread pool:

- No thread creation overhead after initial pool
- Tasks from any level can run on any thread
- Natural work-stealing and load balancing

Running the Solver: run_smart.sh

We use a smart wrapper script to configure OpenMP settings:

```
#!/bin/bash
# Automatically selects optimal OMP settings based on depth

export OMP_DYNAMIC=false

if [ "$DEPTH" -le 3 ]; then
    export OMP_NUM_THREADS=60
    export OMP_PLACES=cores
    export OMP_PROC_BIND=spread
    export OMP_WAIT_POLICY=active
else
    export OMP_NUM_THREADS=60  # or 30 for specific datasets, specifically raisin
fi

# Use NUMA interleaved memory allocation
exec numactl --interleave=all "$@"
```

Usage: `./run_smart.sh ./ConTree -file data.txt -max-depth 3`

Benchmark Results: Overview

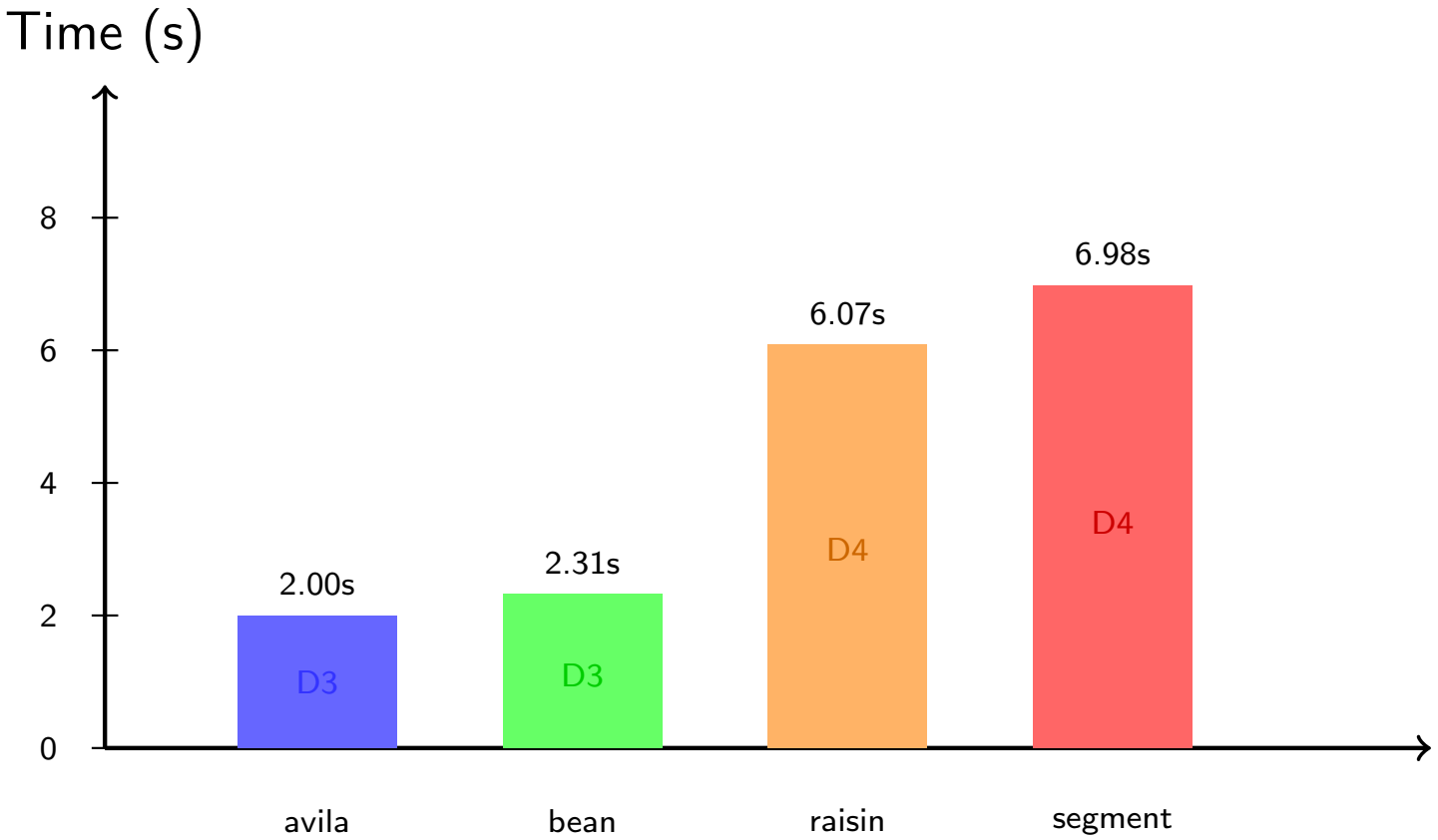
Configuration: 60 threads (30 for raisin), SPREAD binding, 10 runs averaged

Dataset	Depth	Time (s)	Score	Accuracy
avila.txt	3	2.0046	8664	58.5%
bean.txt	3	2.3136	1748	87.2%
raisin.txt	4	6.0740	74	91.8%
segment.txt	4	6.9763	101	95.6%

Key Observations

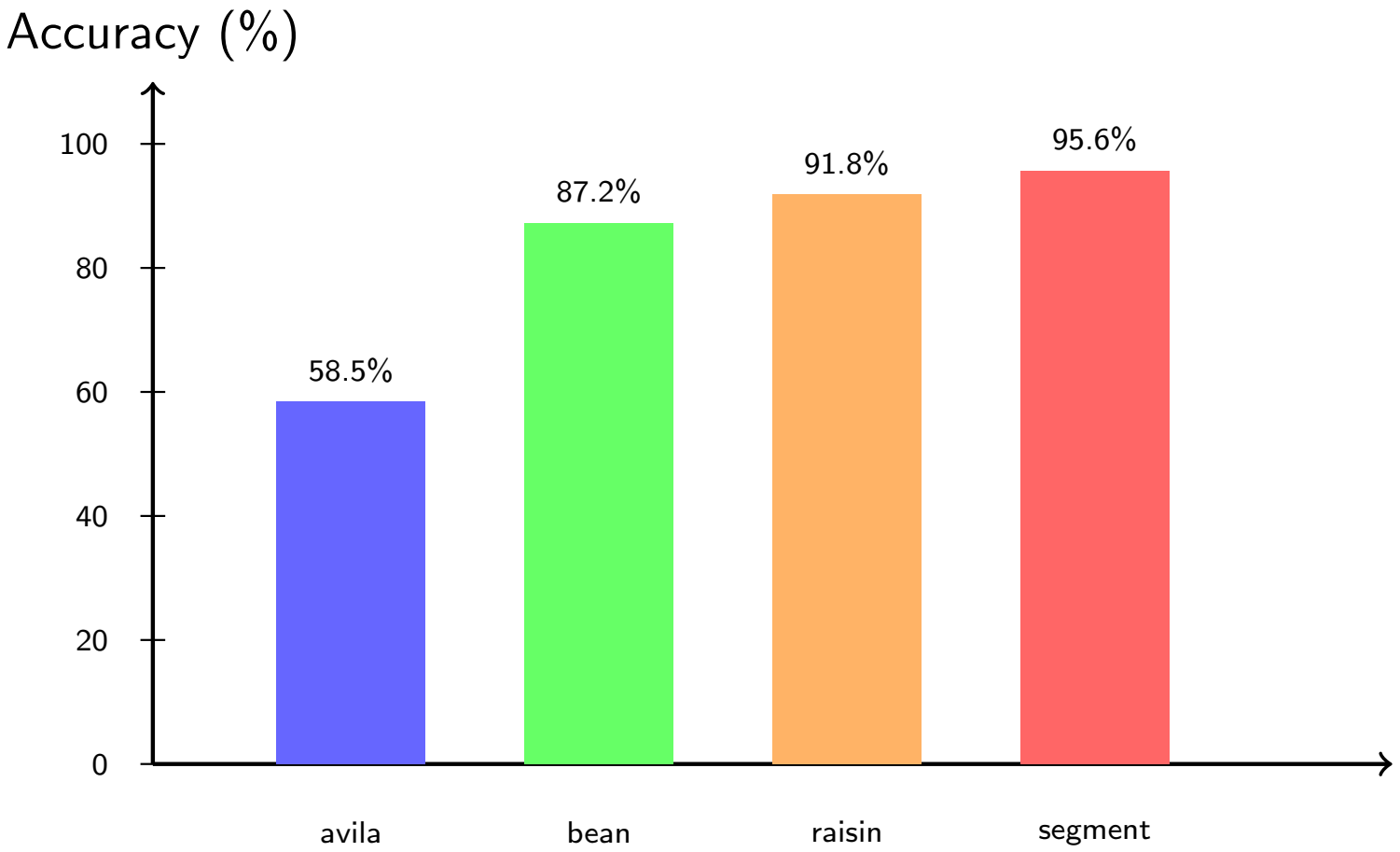
- Depth-4 problems take 2.5-3x longer than depth-3
- Higher accuracy correlates with fewer misclassifications
- Thread count tuned per dataset (raisin benefits from 30 threads)

Benchmark Results: Execution Time



Insight: Depth-4 problems show exponential growth in search space, resulting in 2.5-3x longer execution times compared to depth-3 problems.

Benchmark Results: Accuracy



Insight: All solutions are **provably optimal** for their depth constraint. Lower accuracy on avila reflects dataset complexity, not solver quality.

Key Takeaways

1. **Create parallel region once** at the top level (`main.cpp`)
2. **Use tasks** for all nested parallelism - never nested `parallel` regions
3. **Embarrassingly parallel** parts (left/right subtrees) need no synchronization
4. **Shared mutable state** requires protection (critical sections)
5. **Granularity cutoffs** prevent task overhead for small problems
6. **Atomic** operations for simple counters (lower overhead than critical)

The Big Picture

OpenMP tasks provide a natural fit for recursive tree algorithms - each subproblem becomes a task, and the runtime handles scheduling and load balancing.

Room for Improvement

GPU Parallelization:

- Attempts to parallelize on GPU were unsuccessful, further research is needed

Cache & Memory Optimization:

- Data layout optimization for better cache locality
- NUMA-aware memory allocation beyond `numactl`
- Reduce memory copies when creating dataviews

Algorithmic Improvements:

- Better work-stealing strategies for imbalanced trees
- Dynamic granularity cutoffs based on runtime heuristics
- Priority-based task scheduling (explore promising splits first)
- Redesign the codebase to decrease control divergence

Questions?

Thank you!

Code available on GitHub